

Universidade Federal do Rio de Janeiro
Bacharelado em Ciência da Computação
Disciplina de Oficina de Desenvolvimento de Software I

Projeto da Disciplina - Relatório Parcial (AV1)

Projeto: Agente auxiliar de QA

Grupo 4

Integrantes: Gustavo Teixeira Breda - 121143199
Gabriel de Araujo de Souza - 119053914
João Victor Borges Nascimento - 121064604
Lucas Pinheiro Araújo Silva - 121123995

Professor: Rafael de Mello

1. Introdução

A crescente complexidade dos sistemas de software contemporâneos e a pressão por ciclos de desenvolvimento cada vez mais ágeis intensificaram a necessidade de práticas eficientes de Garantia da Qualidade (QA). Apesar de ser essencial para a validação sistêmica de requisitos, a geração de casos de teste permanece um dos maiores gargalos da área, consistindo em uma atividade custosa, demorada e suscetível a erros humanos. Em abordagens tradicionais, o analista precisa extrair regras implícitas das histórias de usuário e traduzi-las manualmente para cenários, gerando uma sobrecarga técnica.

Este trabalho investiga o uso de IA Generativa para automatizar essa etapa. A ideia central é usar LLMs para receber uma história de usuário em linguagem natural e produzir casos de teste positivos e negativos, verificar ambiguidades nos critérios de aceitação, gerar scripts Selenium/PyTest e consolidar um relatório de cobertura. Para garantir a qualidade das saídas, o pipeline também emprega um modelo como juiz (LLM-as-a-Judge), responsável por revisar se algum cenário relevante foi omitido, além de serem geradores de scripts de testes de ponta-a-ponta (E2E).

Como exemplo concreto, considere a história: "Como usuário cadastrado, quero acessar o sistema com e-mail e senha para consultar minhas solicitações". A partir dessa entrada, o pipeline proposto detecta que critérios como bloqueio por tentativas inválidas e recuperação de senha não foram especificados, gera os casos de teste correspondentes, produz o script de automação e aponta quais pontos ainda exigem decisão humana.

2. Objetivo do Projeto

O objetivo geral do projeto é desenvolver e avaliar um pipeline automatizado, apoiado por IA Generativa, capaz de receber histórias de usuário em linguagem natural, gerar casos de teste funcionais, validar semanticamente esses casos e traduzi-los em scripts executáveis de automação de testes para uma aplicação web de prova de conceito.

O projeto desenvolve um pipeline automatizado que recebe histórias de usuário em linguagem natural e, ao final, entrega scripts de teste executáveis e um relatório de cobertura. O foco não é substituir o analista de QA, e sim eliminar o trabalho mecânico das etapas iniciais: a leitura, extração de regras e escrita dos cenários básicos; deixando para o humano apenas as decisões que realmente exigem julgamento.

2.1 Objetivos Específicos

- Construir prompts estruturados que extraiam casos de teste positivos e negativos a partir de histórias de usuário e critérios de aceitação, incluindo cenários de borda que analistas frequentemente omitem.
- Implementar um componente LLM-as-a-Judge que revise as saídas geradas, identificando omissões, ambiguidades, inconsistências entre critérios e extrapolações fora do escopo da história.
- Converter os casos de teste aprovados pelo juiz em scripts Python/PyTest/Selenium executáveis contra uma aplicação web de prova de conceito.
- Medir o desempenho do pipeline com precisão, recall e F1 dos casos gerados em relação a um gabarito humano, taxa de scripts que executam sem erro e número de inconsistências detectadas pelo juiz em comparação com as identificadas na revisão manual.

As etapas de Engenharia de Software cobertas pelo pipeline são: análise preliminar de requisitos, geração de artefatos de teste, revisão semântica, geração de código de automação e sumarização dos resultados. O ganho esperado é concreto: menos tempo gasto na escrita inicial dos testes e melhor rastreabilidade entre cada critério de aceitação e o caso de teste que o cobre.

3. Trabalhos Relacionados

As pesquisas recentes sobre o uso de LLMs em Engenharia de Software convergem em três preocupações centrais: se modelos conseguem gerar artefatos de teste úteis, quais condições práticas tornam essa geração viável, e como garantir que as saídas sejam confiáveis. Os trabalhos revisados a seguir são organizados em torno dessas três questões.

3.1 O que LLMs conseguem na geração de testes e onde falham

Silva et al. e Quattrocchi et al. atacam diretamente a questão da capacidade dos modelos.

Silva et al. compararam GPT-4o, DeepSeek e Gemini 1.5 Flash na geração de casos de teste a partir de dez histórias de usuário reais de um sistema corporativo brasileiro, usando estratégias zero-shot e one-shot. O resultado é ambíguo: os modelos geram casos úteis, mas sistematicamente omitem requisitos implícitos, exatamente os cenários de borda que mais importam em contexto real.

Quattrocchi et al. chegam a uma conclusão complementar pelo lado dos requisitos: LLMs produzem histórias de usuário fluentes e bem formatadas, mas com menor diversidade e menor aderência a critérios de qualidade do que as escritas por humanos. Juntos, os dois estudos

apontam para o mesmo problema estrutural, a qualidade da entrada determina a qualidade da saída, e mesmo entradas razoáveis produzem saídas com lacunas que precisam ser identificadas e corrigidas.

Nosso projeto responde a isso em duas frentes: um *quality gate* de requisitos antes da geração, que aponta ambiguidades e critérios ausentes na história de usuário, e uma etapa explícita de revisão com LLM-as-a-Judge, com ciclo de reparo para casos reprovados.

3.2 As condições práticas que a literatura frequentemente ignora

Gheventer et al. e Souza et al. trazem uma perspectiva menos comum: o que precisa estar em ordem para que uma solução baseada em IA realmente funcione fora de um ambiente acadêmico controlado.

Gheventer et al. analisam a prontidão industrial de soluções de IA para qualidade de software e concluem que a maioria ainda é protótipo e são dependentes de APIs proprietárias, infraestrutura cara e conhecimento técnico especializado que equipes reais raramente têm. O diagnóstico é direto: há um *last mile problem* entre a ideia acadêmica e um artefato minimamente utilizável.

Souza et al. mostram esse problema de dentro, num relato sobre a evolução de testes funcionais em um sistema público de grande porte. A automação com Selenium só se tornou viável quando a equipe padronizou seletores de interface, antecipou a documentação dos casos no fluxo de desenvolvimento e criou dados de teste estáveis. Sem essas condições, o código gerado simplesmente não executa.

Hernández-Agüero et al. expandem o debate sobre a dependência qualitativa entre inputs e outputs, explorando a capacidade prática de modelos em realizar o saneamento preventivo de requisitos. Através de um ciclo de *human-in-the-loop* aplicado a cenários industriais, os autores validaram o uso de GPT para aprimorar histórias de usuário sob o framework INVEST. A pesquisa evidenciou um alinhamento robusto entre a análise da IA e o crivo de especialistas, mitigando imprecisões linguísticas previamente ao refinamento técnico. Tal evidência sustenta a estruturação da etapa inicial do nosso pipeline: a implementação de um *quality gate* que atua no enriquecimento semântico da história, assegurando que os artefatos de automação subsequentes, cuja execução demanda estabilidade, conforme reiterado por Souza et al., sejam derivados de especificações tecnicamente íntegras e plenamente verificáveis.

Essas duas leituras juntas moldaram decisões concretas do nosso projeto: escopo reduzido e controlado, aplicação de prova de conceito com seletores estáveis (*data-testid*) e uma etapa humana de curadoria antes da execução dos scripts.

3.3 Mecanismos para aumentar a confiabilidade das saídas

Correia et al., Wang et al., Sakib et al. e Sterling e Oliveira abordam, por caminhos diferentes, o problema de como tornar as saídas dos modelos mais confiáveis e rastreáveis.

Correia et al. avaliam RAG no contexto de perguntas técnicas de desenvolvedores sobre o Firefox, comparando GPT-4o com e sem recuperação de contexto. Respostas com RAG foram mais completas e mais aderentes ao domínio, ainda que mais verbosas. A implicação para o projeto é direta: em vez de enviar apenas a história de usuário **ao modelo, o pipeline fornecerá contexto adicional, como glossário do domínio, exemplos de testes aprovados, mapa de telas e seletores para reduzir alucinações e aumentar a aderência ao sistema real.**

3.4 Síntese

O conjunto dos trabalhos revisados indica que a geração automática de casos de teste com LLMs é tecnicamente viável, mas ainda exige três condições que a literatura trata de forma fragmentada: controle da qualidade da entrada, infraestrutura prática mínima e mecanismos explícitos de revisão e rastreabilidade. O presente projeto busca integrar essas três dimensões em um pipeline coeso, ainda que em escala de prova de conceito.

4. Visão da Solução Proposta

O pipeline é composto por cinco agentes especializados e um componente de montagem de contexto, operando em sequência com uma ramificação de reparo. A Figura 1 apresenta a arquitetura completa. A entrada é sempre uma história de usuário acompanhada de critérios de aceitação; a saída final é um relatório de execução com métricas de cobertura.

A arquitetura tem dois princípios centrais. O primeiro é que nenhum artefato avança sem verificação, casos de teste passam pelo Agente 2 antes de virar código, e scripts passam por revisão humana antes de executar. O segundo é que o humano não escreve do zero em nenhuma etapa, ele revisa, aprova ou rejeita o que o pipeline produziu.

O fluxo segue esta sequência: o **Agente 0** verifica a qualidade da história de usuário antes de qualquer geração; o Context Builder agrega informações do projeto, como glossário, exemplos, mapa de telas e seletores, que reduz alucinações; o Agente 1 gera os casos de teste em JSON estruturado; o Agente 2 revisa cobertura, fidelidade ao requisito e automatizabilidade; casos reprovados retornam ao Agente 1 para reparo com limite de iterações; casos aprovados passam por revisão humana antes de chegar ao Agente 3, que gera os scripts Selenium/PyTest; após execução, o Agente de Sumarização consolida o relatório final.

A escolha do modelo e do framework de orquestração será feita durante a implementação, guiada por custo, latência e disponibilidade de acesso pela equipe. Essa decisão não altera a arquitetura do pipeline, os prompts e o fluxo entre agentes são independentes do modelo utilizado.

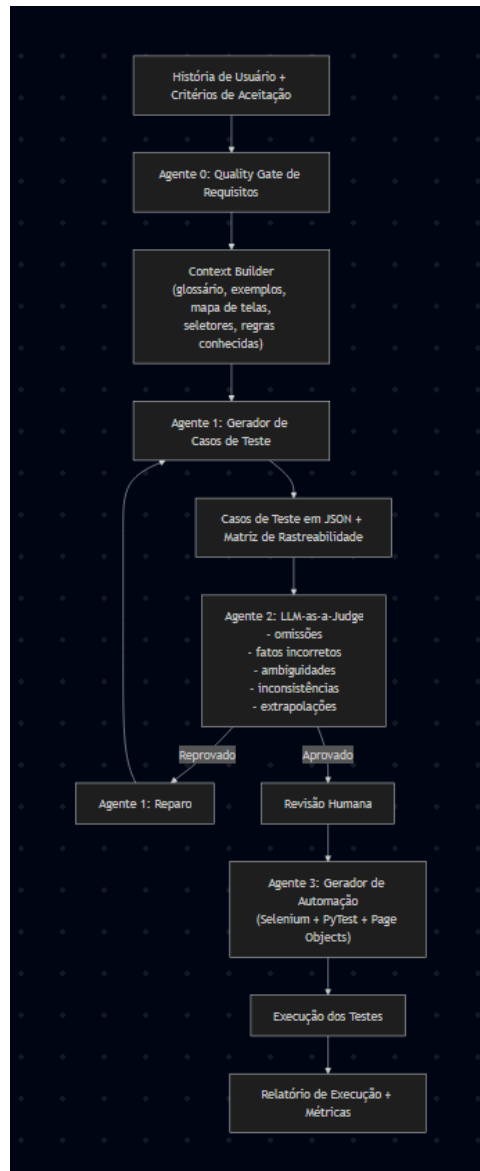


Figura 1- Representação Arquitetural da Solução

5. Fluxo Geral

A entrada do pipeline será uma história de usuário acompanhada, sempre que possível, por critérios de aceitação. Antes de gerar testes, o Agente 0 fará uma análise de qualidade da história, apontando problemas como requisitos vagos, ausência de resultados esperados, regras

de negócio incompletas, falta de dados de entrada ou critérios não verificáveis. Essa etapa foi adicionada porque estudos sobre histórias de usuário geradas ou avaliadas por LLMs mostram que a qualidade semântica dos requisitos é um fator crítico para a utilidade dos artefatos derivados.

Em seguida, o Context Builder reunirá informações auxiliares do projeto, como glossário, exemplos de testes, padrões de escrita, dados de teste, nomes de telas e seletores HTML. Esta etapa é inspirada em abordagens RAG, nas quais o modelo recebe informações específicas do projeto antes de gerar uma resposta. Correia et al. mostram que RAG pode tornar respostas técnicas mais completas ao recuperar documentação e código do projeto, embora seja necessário controlar a verbosidade.

O Agente 1 será responsável por gerar os casos de teste funcionais. Ele deverá aplicar técnicas como partição de equivalência, valores-limite, fluxos positivos, fluxos negativos, permissões de usuário e estados inválidos. A saída será estruturada em JSON para facilitar validação automática e geração de código. Cada caso de teste deverá conter identificador, objetivo, requisito coberto, pré-condições, dados de entrada, passos, resultado esperado, tipo de cenário e prioridade.

O Agente 2 atuará como LLM-as-a-Judge. Ele receberá a história original, os critérios de aceitação, o contexto e os casos de teste gerados. Sua função será verificar se os testes cobrem os critérios de aceitação, se incluem fatos não autorizados, se deixam de cobrir cenários relevantes e se há inconsistências entre passos e resultados esperados. Casos reprovados voltarão ao Agente 1 para reparo. Essa etapa endereça diretamente a limitação observada por Silva et al., que identificaram omissões como uma falha frequente na geração de testes por LLMs.

O Agente 3 gerará scripts de automação utilizando Python, PyTest e Selenium. Sempre que possível, a aplicação web de prova de conceito será desenvolvida com seletores estáveis, como data-testid, reduzindo a fragilidade dos testes automatizados. Essa decisão é motivada pelo relato de Souza et al., no qual práticas ruins de nomeação e identificadores aleatórios foram apontadas como barreiras técnicas para automação funcional.

6. Modelos e Técnicas

Os modelos considerados para implementação são GPT-5, Claude Sonnet 4.6 e Gemini 2.5 Pro, ou equivalentes disponíveis no momento da execução. A escolha final dependerá de custo, latência, capacidade de seguir instruções complexas e disponibilidade de acesso pela equipe. Vale notar que Silva et al. observaram um platô de desempenho entre modelos na geração de casos de teste, e diferenças de qualidade entre modelos top-tier tendem a ser menores do que

diferenças provocadas pelo desenho do prompt. Por isso, o projeto não aposta em um modelo específico como fator de sucesso; ele buscará compensar limitações com bom desenho de prompt, contexto estruturado e revisão em múltiplas etapas.

As principais técnicas aplicadas serão:

- **Persona prompting** - o modelo é instruído a atuar como especialista sênior de QA, o que tende a produzir saídas mais estruturadas e criteriosas.
- **Context injection** - história de usuário, critérios de aceitação, glossário e mapa de interface são fornecidos explicitamente no prompt, reduzindo suposições do modelo sobre o domínio.
- **Few-shot prompting** - quando houver exemplos de casos de teste já aprovados, eles serão incluídos para calibrar formato e nível de detalhe esperado.
- **Saída estruturada em JSON** - obrigatória para validação automática da estrutura e geração de código pelo Agente 3.
- **LLM-as-a-Judge** - revisão independente de cobertura, consistência e aderência ao domínio antes da automação.
- **Ciclo de reparo** - casos reprovados retornam ao Agente 1 com diagnóstico, com limite de iterações para evitar loops.
- **Geração multi-candidato** - quando viável, o Agente 1 produz mais de uma versão dos casos e o juiz seleciona a melhor.

7. Papel dos Desenvolvedores e Testadores

O pipeline reduz trabalho mecânico, não responsabilidade. As decisões que exigem julgamento permanecem com os integrantes do grupo. Concretamente, o papel humano envolve:

1. Escrever ou revisar histórias de usuário e critérios de aceitação antes de alimentar o pipeline.
2. Criar o glossário e o mapa de interface da aplicação de prova de conceito.
3. Revisar os alertas do Agente 0 e decidir se a história avança ou precisa de refinamento.
4. Aprovar ou rejeitar os casos de teste propostos pelo Agente 1 após revisão do juiz.
5. Revisar os scripts gerados pelo Agente 3 antes da execução.
6. Executar os scripts e comparar resultados com um oráculo humano.
7. Registrar limitações, erros e aprendizados ao longo do processo.

O que muda com o pipeline é a velocidade e o ponto de partida, já que o analista revisa e decide, em vez de escrever do zero.

8. Engenharia de Prompt

A seguir estão as versões iniciais dos prompts a serem utilizados.

8.1 Análise de Qualidade da História de Usuário

Você é um Analista Sênior de Requisitos e Qualidade de Software.

Sua tarefa é avaliar uma História de Usuário e seus Critérios de Aceitação antes da geração de casos de teste. Seja criterioso: uma história bem escrita pode ainda ter critérios ausentes ou dados não especificados. Não aprove por padrão – aprove porque verificou cada item abaixo.

```
<user_story>
{user_story}
</user_story>
```

```
<acceptance_criteria>
{acceptance_criteria}
</acceptance_criteria>
```

```
<system_context>
{system_context}
</system_context>
```

Verifique cada item e registre problemas encontrados:

1. Existem ambiguidades ou termos vagos que precisam de definição?
2. Os critérios de aceitação são verificáveis e mensuráveis?
3. Há critérios de aceitação ausentes para cobrir os fluxos negativos e de borda esperados?
4. Existem regras de negócio contraditórias entre si ou com o contexto?
5. Há dados de entrada, estados do sistema ou pré-condições necessários não foram especificados?
6. A história, como está escrita, permite derivar casos de teste funcionais sem que o analista precise inventar regras?

Observação sobre formato: se a história não seguir o padrão "Como [papel], quero [ação], para [benefício]", registre como

observação, não como problema bloqueante - o conteúdo importa mais que o formato.

Regras obrigatórias:

- Não invente regras de negócio ausentes.
- Se algo não estiver claro, formule uma pergunta objetiva para o Product Owner.
- Se não houver problemas, retorne "problemas": [] e status "APROVADA".

Retorne estritamente em JSON, sem texto fora do JSON:

```
{
  "status": "APROVADA" | "PRECISA_DE_ESCLARECIMENTO",
  "derivavel": true | false,
  "justificativa_derivabilidade": "...",
  "observacao_formato": "...",
  "problemas": [
    {
      "tipo": "ambiguidade" | "omissao" | "inconsistencia" |
        "critério_nao_testavel" | "falta_de_dado",
      "descricao": "...",
      "impacto_em_testes": "...",
      "pergunta_para_o_product_owner": "..."
    }
  ],
  "recomendacao": "..."
}
```

8.2 Geração de Casos de Teste

Você é um Especialista Sênior de Qualidade de Software com conhecimento em ISTQB, testes funcionais, partição de equivalência, análise de valor-limite e testes de aplicações web.

Sua tarefa é gerar casos de teste funcionais a partir da História de Usuário abaixo. Seja exaustivo nos cenários negativos e de borda - esses são os que analistas humanos mais frequentemente omitem.

```
<user_story>
{user_story}
</user_story>
```

```
<acceptance_criteria>
{acceptance_criteria}
</acceptance_criteria>
```

```
<system_context>
{system_context}
</system_context>
```

```
<domain_glossary>
{domain_glossary}
</domain_glossary>
```

```
<exemplos_de_testes_aprovados>
{few_shot_examples}
</exemplos_de_testes_aprovados>
```

Regras obrigatórias:

1. Gere cenários positivos, negativos e de borda para cada critério de aceitação.
2. Aplique partição de equivalência: para cada campo de entrada, identifique classes válidas e inválidas e gere ao menos um caso por classe.
3. Não crie regras de negócio ausentes na história, nos critérios ou no contexto. Se precisar assumir algo, registre em "observacoes".
4. Cada caso de teste deve cobrir ao menos um critério de aceitação – nenhum caso pode ficar sem rastreabilidade.
5. Os passos devem ser objetivos o suficiente para virar automação Selenium sem ambiguidade – evite passos como "preencha os dados corretamente".
6. Dados de teste devem ser concretos – use valores reais, não placeholders como "email@exemplo.com" ou "senha123".
7. Retorne apenas JSON válido, sem texto fora do JSON.

Antes de retornar, verifique internamente:

- Todo critério de aceitação tem ao menos um caso positivo e um negativo?
- Há cenários de borda para campos numéricos, datas e limites de caracteres?
- Algum passo está vago demais para ser automatizado?

Retorne estritamente no formato abaixo:

```

{
  "test_cases": [
    {
      "id": "CT-001",
      "titulo": "...",
      "objetivo": "...",
      "criterios_cobertos": ["CA-01"],
      "tipo": "positivo | negativo | borda | permissao | regressao",
      "prioridade": "alta | media | baixa",
      "pre_condicoes": ["..."],
      "dados_de_teste": {
        "campo": "valor_concreto"
      },
      "passos": [
        "Acesse a URL /login",
        "Preencha o campo [data-testid='email'] com {email}",
        "..."
      ],
      "resultado_esperado": "...",
      "automatizavel": true | false,
      "observacoes": "..."
    }
  ],
  "matriz_rastreabilidade": [
    {
      "criterio": "CA-01",
      "casos_de_teste": ["CT-001", "CT-002"]
    }
  ],
  "alertas": [
    "Critério CA-03 não tinha dados suficientes – assumiu-se que..."
  ]
}

```

8.3 LLM-as-a-Judge

Você é um Revisor Sênior de Qualidade de Software. Sua tarefa é avaliar se os casos de teste gerados estão corretos, completos e fiéis à História de Usuário original.

Atenção: avalie com base exclusivamente no que está escrito na história, nos critérios de aceitação e no contexto. Não aprove um caso porque ele parece razoável - aprove porque ele é rastreável à

especificação.

Não reprove por estilo ou preferência pessoal.

```
<user_story>
{user_story}
</user_story>
```

```
<acceptance_criteria>
{acceptance_criteria}
</acceptance_criteria>
```

```
<system_context>
{system_context}
</system_context>
```

```
<generated_test_cases>
{generated_test_cases_json}
</generated_test_cases>
```

Para cada caso de teste, verifique:

1. Cobertura: o caso cobre ao menos um critério de aceitação listado? Se sim, qual? Se não, está justificado?
2. Fato incorreto: o caso assume alguma regra de negócio ausente na história, nos critérios ou no contexto? Cite a evidência.
3. Omissão: existe cenário relevante que deveria existir e não foi gerado? Considere fluxos negativos, bordas, permissões e estados inválidos.
4. Ambiguidade: algum passo ou resultado esperado está vago a ponto de impedir a automação? Exemplos: "preencha corretamente", "verifique o resultado", "clique no botão certo".
5. Inconsistência: algum caso contradiz a história, os critérios ou outro caso do mesmo conjunto?
6. Automatizabilidade: os passos são objetivos e concretos o suficiente para virar código Selenium sem interpretação adicional?

Regras de decisão:

- Status APROVADO: nenhum problema de fato incorreto ou inconsistência;
cobertura aceitável; passos automatizáveis.
- Status REPROVADO: qualquer fato incorreto, inconsistência grave, cobertura abaixo de 70% dos critérios, ou passos sistematicamente vagos.

- Casos individuais podem ser aprovados mesmo se o conjunto for reprovado - indique quais estão aprovados e quais precisam de reparo.

Pontue cada dimensão de 0 a 10, onde:

- 0-4: insatisfatório
- 5-7: aceitável com ressalvas
- 8-10: satisfatório

Retorne estritamente em JSON, sem texto fora do JSON:

```
{
  "status_geral": "APROVADO" | "REPROVADO",
  "pontuacao": {
    "cobertura": 0,
    "fidelidade_ao_requisito": 0,
    "clareza": 0,
    "automatizabilidade": 0
  },
  "casos_aprovados": ["CT-001", "CT-002"],
  "casos_reprovados": ["CT-003"],
  "problemas": [
    {
      "caso_de_teste": "CT-003",
      "tipo": "omissao" | "fato_incorreto" | "ambiguidade" |
        "inconsistencia" | "baixa_automatizabilidade",
      "descricao": "...",
      "evidencia_na_historia": "trecho exato da história ou critério
        que contradiz ou ausenta a regra",
      "acao_recomendada": "..."
    }
  ],
  "cenarios_omitidos_sugeridos": [
    {
      "descricao": "...",
      "criterio_relacionado": "CA-...",
      "justificativa": "por que esse cenário é necessário"
    }
  ],
  "decisao": "..."
}
```

8.4 Reparo dos Casos de Teste

Você é um Especialista de QA responsável por corrigir casos de teste reprovados por uma revisão semântica.

Sua tarefa é aplicar exatamente as correções indicadas pelo relatório do juiz - nem mais, nem menos. Não reescreva casos aprovados. Não invente regras de negócio ausentes na história ou nos critérios.

```
<user_story>
{user_story}
</user_story>
```

```
<acceptance_criteria>
{acceptance_criteria}
</acceptance_criteria>
```

```
<system_context>
{system_context}
</system_context>
```

```
<generated_test_cases>
{generated_test_cases_json}
</generated_test_cases>
```

```
<judge_report>
{judge_report_json}
</judge_report>
```

Instruções de reparo:

1. Para cada caso em "casos_reprovados" do relatório:
 - Aplique a "acao_recomendada" correspondente.
 - Mantenha o ID original se for correção; use novo ID (CT-0XX) se for substituição completa.
 - Registre em "correcao_aplicada" o que foi alterado e por quê.
2. Para cada item em "cenarios_omitidos_sugeridos" do relatório:
 - Crie um novo caso de teste com novo ID.
 - Registre em "correcao_aplicada": "cenário omitido adicionado conforme sugestão do juiz".
3. Casos em "casos_aprovados" do relatório:

- Retorne-os sem alteração.
- 4. Atualize a matriz de rastreabilidade para refletir todos os casos, incluindo os novos.
- 5. Se uma correção indicada pelo juiz exigir inventar uma regra de negócio ausente, não aplique a correção – registre em "alertas" explicando o motivo.

Retorne estritamente em JSON, sem texto fora do JSON:

```
{
  "test_cases": [
    {
      "id": "CT-001",
      "titulo": "...",
      "objetivo": "...",
      "criterios_cobertos": ["CA-01"],
      "tipo": "positivo | negativo | borda | permissao | regressao",
      "prioridade": "alta | media | baixa",
      "pre_condicoes": ["..."],
      "dados_de_teste": {
        "campo": "valor_concreto"
      },
      "passos": ["..."],
      "resultado_esperado": "...",
      "automatizavel": true | false,
      "correcao_aplicada": "descrição do que foi corrigido,
                           ou 'nenhuma – caso aprovado'",
      "observacoes": "..."
    }
  ],
  "matriz_rastreabilidade": [
    {
      "criterio": "CA-01",
      "casos_de_teste": ["CT-001", "CT-002"]
    }
  ],
  "alertas": [
    "Correção do CT-003 não aplicada: exigiria assumir regra de
    negócio sobre X, ausente na história."
  ]
}
```


8.5 Geração de Código Selenium/PyTest

Você é um Engenheiro de Automação de Testes especializado em Python, PyTest e Selenium.

Sua tarefa é gerar scripts de automação para os casos de teste aprovados abaixo. Use apenas os seletores presentes no mapa de telas - não invente seletores. Se um seletor necessário estiver ausente, registre em "pendencias_de_automacao" e pule o caso.

```
<approved_test_cases>
{approved_test_cases_json}
</approved_test_cases>
```

```
<ui_map>
{ui_map}
</ui_map>
```

Regras obrigatórias:

1. Use Python com PyTest e Selenium.
2. Aplique Page Object Model: cada tela vira uma classe em pages.py, com métodos que encapsulam as ações - não acesse seletores diretamente nas funções de teste.
3. Priorize seletores data-testid. Use CSS ou XPath apenas quando data-testid não estiver disponível no mapa.
4. Não use time.sleep() - use WebDriverWait com condições explícitas (visibility_of_element_located, element_to_be_clickable).
5. Cada caso de teste vira uma função de teste nomeada test_{id_do_caso}_{descricao_curta}.
6. Inclua um comentário no início de cada função indicando o ID e o objetivo do caso de teste.
7. Dados de teste devem vir de fixtures ou do próprio JSON de casos - não hardcode valores dentro das funções de teste.
8. Retorne três blocos separados e completos: conftest.py, pages.py e test_{nome_funcionalidade}.py.

Retorne no formato abaixo:

```
{
  "arquivos": {
    "conftest.py": "# conteúdo completo do arquivo",
    "pages.py": "# conteúdo completo do arquivo",
```

```

    "test_{nome_funcionalidade}.py": "# conteúdo completo do arquivo"
  },
  "pendencias_de_automacao": [
    {
      "caso_de_teste": "CT-...",
      "motivo": "seletor do campo X ausente no mapa de telas",
      "acao_recomendada": "adicionar data-testid='...' ao elemento"
    }
  ]
}

```

8.6 Sumarização da Execução

Você é um Analista de QA responsável por interpretar resultados de execução de testes automatizados e produzir um relatório que auxilie a tomada de decisão humana.

Sua tarefa não é apenas classificar falhas – é distinguir o que é problema do sistema, o que é problema do teste e o que é problema de ambiente. Seja preciso: uma falha de seletor não é um bug do sistema.

```

<approved_test_cases>
{approved_test_cases_json}
</approved_test_cases>

```

```

<pytest_output>
{pytest_output}
</pytest_output>

```

```

<selenium_logs>
{selenium_logs}
</selenium_logs>

```

```

<error_screenshots_or_messages>
{error_evidence}
</error_screenshots_or_messages>

```

Instruções de análise:

1. Para cada caso de teste, determine o resultado: passou, falhou ou não executou.

2. Para cada falha, classifique a causa provável:
 - "erro_no_sistema": o sistema se comportou diferente do esperado – possível bug real.
 - "erro_no_teste": o script está incorreto, passo errado ou resultado esperado mal definido.
 - "seletor_invalido": elemento não encontrado – seletor desatualizado ou ausente na tela.
 - "dado_invalido": dado de teste incompatível com o estado atual do sistema.
 - "ambiente": falha de infraestrutura, timeout de rede, banco indisponível.
3. Para cada critério de aceitação, determine se foi coberto e se os casos que o cobrem passaram ou falharam.
4. Ao final, indique quais falhas exigem ação imediata e quais podem aguardar – isso orienta a priorização da equipe.

Regras:

- Use apenas as evidências fornecidas. Não assuma causas sem base nos logs ou mensagens de erro.
- Se a causa não puder ser determinada com os dados disponíveis, classifique como "indeterminado" e indique o que falta para diagnosticar.
- Não repasse ao relatório informações que não estejam nos inputs.

Retorne estritamente em JSON, sem texto fora do JSON:

```
{
  "resumo": {
    "total": 0,
    "passaram": 0,
    "falharam": 0,
    "nao_executaram": 0,
    "quebraram_por_seletor": 0,
    "quebraram_por_regra_de_negocio": 0,
    "quebraram_por_ambiente": 0
  },
  "falhas": [
    {
      "caso_de_teste": "CT-...",
      "tipo_provavel": "erro_no_sistema | erro_no_teste |
```

```

        seletor_invalido | dado_invalido |
        ambiente | indeterminado",
    "descricao": "...",
    "evidencia": "trecho exato do log ou mensagem de erro",
    "acao_recomendada": "...",
    "prioridade": "imediata | pode_aguardar"
  }
],
"cobertura_por_criterio": [
  {
    "criterio": "CA-...",
    "casos_relacionados": ["CT-001", "CT-002"],
    "status": "coberto_e_passou | coberto_e_falhou | nao_executado
| nao_coberto"
  }
],
"alertas_de_qualidade": [
  "Critério CA-02 foi coberto apenas por casos negativos –
  nenhum fluxo positivo foi executado.",
  "..."
],
"proximos_passos": [
  {
    "acao": "...",
    "responsavel_sugerido": "dev | qa | devops",
    "urgencia": "imediata | proxima_sprint | backlog"
  }
]
}

```

9. Visão de Avaliação

A avaliação do pipeline será realizada por meio de uma prova de conceito com escopo controlado, composta por uma aplicação web desenvolvida pelo grupo e um conjunto fixo de histórias de usuário. O objetivo é verificar se o pipeline consegue gerar casos de teste úteis, identificar seus pontos de falha e quantificar o esforço humano necessário para supervisionar as saídas.

9.1 Aplicação de Prova de Conceito

A aplicação será desenvolvida pelo grupo usando FastAPI no backend e Angular no frontend. O escopo foi definido para cobrir os tipos de fluxo que o pipeline precisa ser capaz de testar:

autenticação, permissões por perfil, validação de campos, persistência e operações com regras de negócio explícitas.

As funcionalidades previstas são login, cadastro de usuário, criação de solicitação de atendimento, listagem e busca de solicitações, e edição ou cancelamento conforme permissão do perfil. Os elementos interativos serão marcados com *data-testid* desde o início do desenvolvimento, evitando o problema de seletores instáveis relatado por Souza et al.

9.2 Métricas de Avaliação

As métricas foram organizadas em três grupos: qualidade dos casos de teste gerados, qualidade da automação e eficácia dos componentes do pipeline. A abordagem combina métricas quantitativas com análise qualitativa de defeitos semânticos, seguindo a metodologia de Silva et al., o que permitirá comparação direta com os resultados daquele estudo.

9.2.1 Qualidade dos casos de teste

- **Precisão:** proporção de casos gerados pelo pipeline que foram considerados corretos pelo oráculo humano.
- **Recall:** proporção de casos esperados no oráculo que foram cobertos pelo pipeline.
- **F1-score:** média harmônica entre precisão e recall.
- **Taxa de omissão:** quantidade de cenários esperados pelo oráculo que não foram gerados.
- **Taxa de fato incorreto:** quantidade de casos que assumem regra de negócio ausente na história ou nos critérios.
- **Cobertura dos critérios de aceitação:** percentual de critérios com ao menos um caso de teste aprovado associado.

9.2.2 Qualidade da automação

- **Taxa de scripts executáveis:** percentual de scripts gerados que executam sem erro de sintaxe ou falha de seletor.
- **Taxa de sucesso funcional:** percentual de scripts que passam quando a funcionalidade está correta no sistema.

9.2.3 Eficácia dos componentes do pipeline

- **Precisão do juiz:** proporção de problemas reportados pelo Agente 2 que foram confirmados pela revisão humana.
- **Recall do juiz:** proporção de problemas identificados pela revisão humana que o Agente 2 também detectou.

- **Esforço percebido:** tempo registrado pelo analista na revisão dos artefatos gerados pelo pipeline, comparado ao tempo que a criação manual levou para as mesmas histórias.

10. Histórias de Usuário Iniciais

As histórias de usuário a seguir foram elaboradas com critérios de aceitação explícitos, pois o pipeline proposto depende deles como entrada estruturada para o Agente 0 e o Agente 1. Cada história recebeu três critérios cobrindo, respectivamente, o caminho feliz, uma regra de rejeição e um comportamento de borda ou segurança.

Essa estrutura garante que o Agente 1 tenha material suficiente para derivar casos positivos, negativos e de borda, conforme previsto em seu prompt. Os critérios foram redigidos com valores concretos, mensagens específicas e estados nomeados para que o próprio Agente 0 não os reprove por ambiguidade ou por não serem verificáveis. A numeração adotada (CA-XX.Y) segue o padrão esperado pelo campo `critérios_cobertos` do Agente 1, facilitando a geração automática da matriz de rastreabilidade.

10.1 US-01 - Login

Como usuário cadastrado, quero acessar o sistema com e-mail e senha para consultar minhas solicitações.

Critérios de aceitação:

- CA-01.1: O sistema deve autenticar o usuário quando o e-mail estiver cadastrado e a senha corresponder à armazenada, redirecionando-o para a tela de listagem de solicitações.
- CA-01.2: O sistema deve exibir a mensagem "E-mail ou senha inválidos" quando as credenciais não corresponderem a um usuário cadastrado, sem indicar qual dos dois campos está incorreto.
- CA-01.3: O sistema deve bloquear novas tentativas de login por 60 segundos após cinco falhas consecutivas com o mesmo e-mail.

10.2 US-02 - Cadastro de usuário

Como novo usuário, quero criar uma conta informando nome, e-mail e senha para acessar o sistema.

Critérios de aceitação:

- CA-02.1: O sistema deve aceitar cadastros em que o nome tenha entre 3 e 80 caracteres, o e-mail siga o formato padrão (texto@dominio.tld) e a senha tenha no mínimo 8 caracteres, incluindo ao menos uma letra e um número.
- CA-02.2: O sistema deve rejeitar o cadastro quando o e-mail informado já estiver registrado, exibindo a mensagem "E-mail já cadastrado".
- CA-02.3: Após cadastro bem-sucedido, o sistema deve exibir mensagem de confirmação e redirecionar o usuário para a tela de login.

10.3 US-03 - Criação de solicitação

Como usuário autenticado, quero criar uma solicitação de atendimento com título, descrição e prioridade para que a equipe possa analisá-la.

Critérios de aceitação:

- CA-03.1: O sistema deve permitir a criação de solicitação quando o título tiver entre 5 e 100 caracteres, a descrição entre 10 e 500 caracteres e a prioridade for um dos valores "baixa", "média" ou "alta".
- CA-03.2: O sistema deve atribuir automaticamente o status "aberta" e a data de criação à nova solicitação, vinculando-a ao usuário autenticado.
- CA-03.3: O sistema deve rejeitar a criação quando algum campo obrigatório estiver vazio, sinalizando o campo correspondente com mensagem de erro específica.

10.4 US-04 - Busca de solicitações

Como usuário autenticado, quero filtrar minhas solicitações por status e prioridade para localizar rapidamente registros importantes.

Critérios de aceitação:

- CA-04.1: O sistema deve exibir, por padrão, apenas as solicitações criadas pelo usuário autenticado, ordenadas da mais recente para a mais antiga.
- CA-04.2: O sistema deve permitir a combinação de filtros por status ("aberta", "em análise", "finalizada", "cancelada") e por prioridade ("baixa", "média", "alta"), atualizando a listagem conforme os filtros aplicados.
- CA-04.3: O sistema deve exibir a mensagem "Nenhuma solicitação encontrada" quando a combinação de filtros não retornar resultados, sem ocultar os controles de filtro.

10.5 US-05 - Cancelamento de solicitação

Como usuário autenticado, quero cancelar uma solicitação ainda não finalizada para evitar que ela continue em análise indevidamente.

Critérios de aceitação:

- CA-05.1: O sistema deve permitir o cancelamento apenas de solicitações com status "aberta" ou "em análise" e que pertençam ao usuário autenticado.
- CA-05.2: O sistema deve solicitar confirmação ao usuário antes de efetivar o cancelamento, exibindo o título da solicitação na mensagem.
- CA-05.3: Após o cancelamento, o sistema deve atualizar o status da solicitação para "cancelada", registrar a data do cancelamento e impedir edições posteriores.

11. Limitações Esperadas

Quatro limitações estruturais do pipeline foram identificadas antes da execução e merecem registro explícito.

A mais fundamental é que LLMs podem gerar saídas plausíveis e incorretas ao mesmo tempo, e casos de teste bem formatados que assumem regras de negócio não especificadas ou omitem cenários implícitos. Essa limitação é documentada por Silva et al. e não é eliminada pelo ciclo de reparo; ela é apenas reduzida. O Agente 2 também está sujeito ao mesmo problema: um juiz baseado em LLM pode aprovar casos problemáticos por superficialidade ou reprovar casos válidos por viés de estilo, conforme alertam as desvantagens do LLM-as-a-Judge discutidas na disciplina.

A terceira limitação é técnica: os scripts Selenium gerados pelo Agente 3 dependem da existência de seletores estáveis na interface. Qualquer mudança de layout que altere os atributos *data-testid* invalida os scripts sem que o pipeline detecte automaticamente. Por isso, a manutenção do mapa de interface é responsabilidade contínua da equipe, não uma tarefa pontual.

A quarta é de escopo: cinco histórias de usuário em uma aplicação controlada não são representativas de sistemas industriais complexos. Os resultados da avaliação indicarão o comportamento do pipeline nesse contexto específico extrapolações para sistemas de maior porte exigiriam estudos adicionais.

Nenhuma dessas limitações inviabiliza a prova de conceito. Elas delimitam o que pode ser concluído a partir dela.

12. Referências

- Silva, L. et al. Investigating the Use of Large Language Models (LLMs) in Test Case Generation. Material anexado ao projeto.
- Gheventer, A. et al. Generative AI Solutions for Software Quality: Assessing Industrial Readiness. Material anexado ao projeto.
- Souza, B. P. et al. A Journey of Functional Test Evolution in the Public Sector. Material anexado ao projeto.
- Correia, J. et al. A Comparison of Conversational Models and Humans in Answering Technical Questions: the Firefox Case. Material anexado ao projeto.
- Qin, P.; Zhang, R.; Cao, Q.; Xie, P. DAJ: Data-Rewighted LLM Judge for Test-Time Scaling in Code Generation. arXiv, 2026.
- Wang, Y. et al. Co-Evolving LLM Coder and Unit Tester via Reinforcement Learning. NeurIPS, 2025.
- Quattrocchi, G. et al. Can LLMs Generate User Stories and Assess Their Quality? IEEE Transactions on Software Engineering, 2026.
- Sakib, S. et al. From Reviews to Requirements: Can LLMs Generate Human-Like User Stories? arXiv, 2026.
- Sterling, L.; Oliveira, E. Hybrid Intelligence in Requirements Education: Preserving Student Agency in Refining User Stories with Generative AI. Information, 2026.
- Hernández-Agüero, E.; Quesada-López, C.; Chaves-Sánchez, J. P. LLM-Assisted INVEST Evaluation and Improvement of User Stories: An Industrial Replication Study.